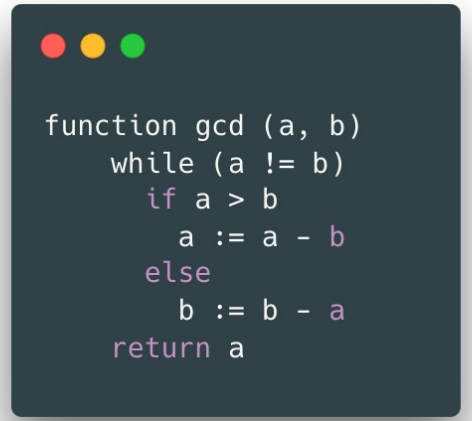


Analiza Algoritmilor

Algorithms & Complexity Theory

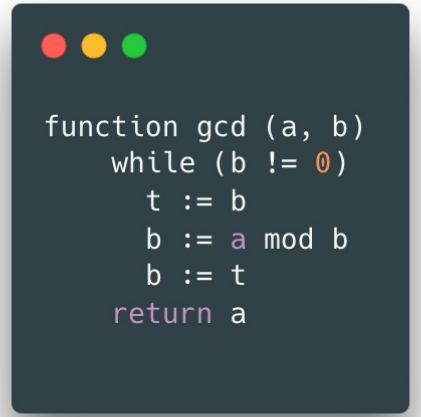
Matei Popovici

Which algorithm is better ?



```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

Alg A



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

Alg B

How to we evaluate the performance of an algorithm?

Number of lines of (pseudo)code

How to we evaluate the performance of an algorithm?

We implement the algorithm in some programming language

We measure the execution time on very big inputs

How to we evaluate the performance of an algorithm?

We determine how the number of performed instructions scale with respect to the size of the input.

How to we evaluate the performance of an algorithm?

We determine how the number of performed instructions scale with respect to the size of the worst-case input.

How to we evaluate the performance of an algorithm?

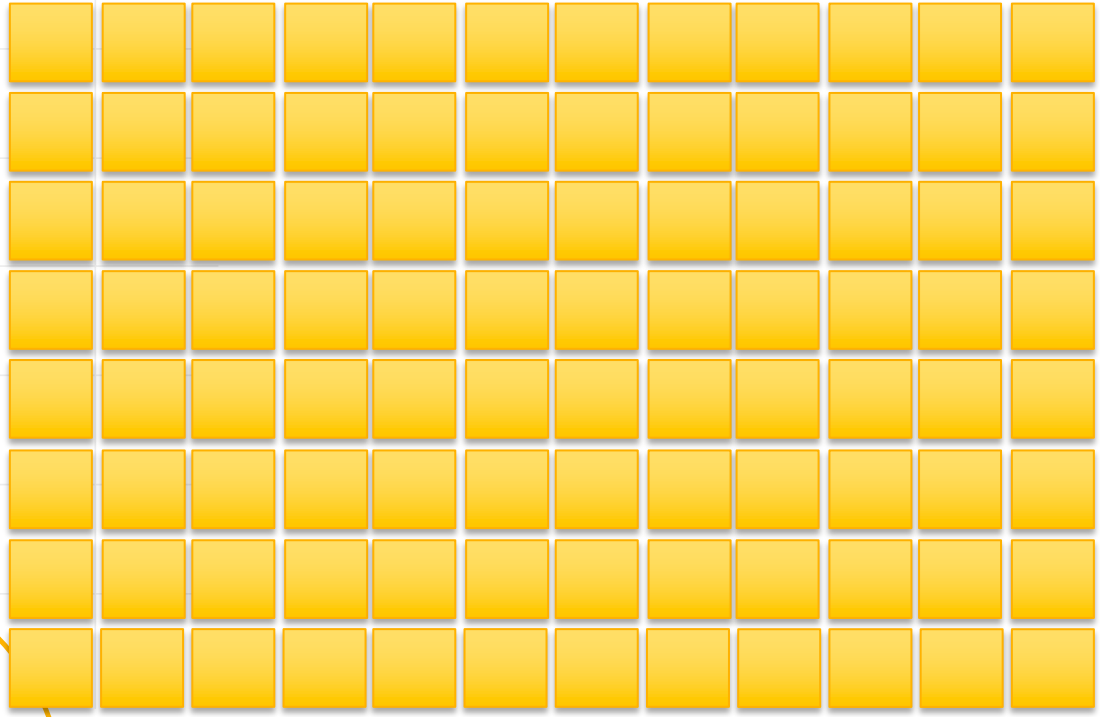
We determine how the consumed memory scales with respect to the size of the worst-case input.

Two variants of Euclid's algorithm

```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

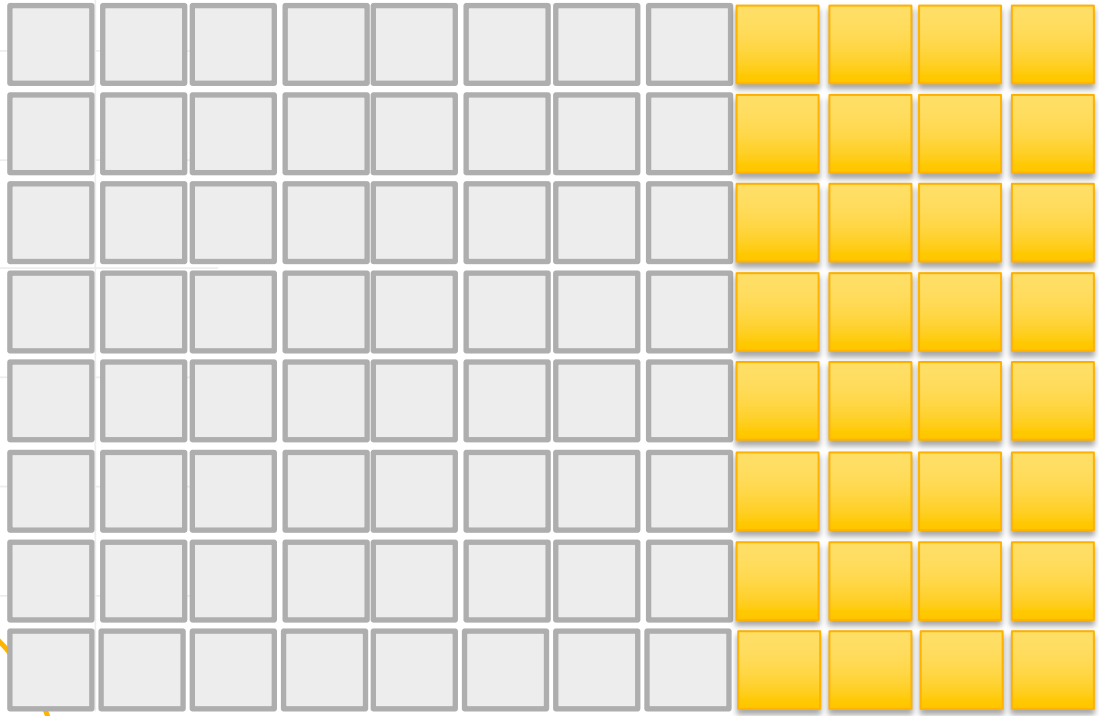
```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

finding gcd(8,12)



What is the largest square size S that can be used to pave a 8 x 12 surface?

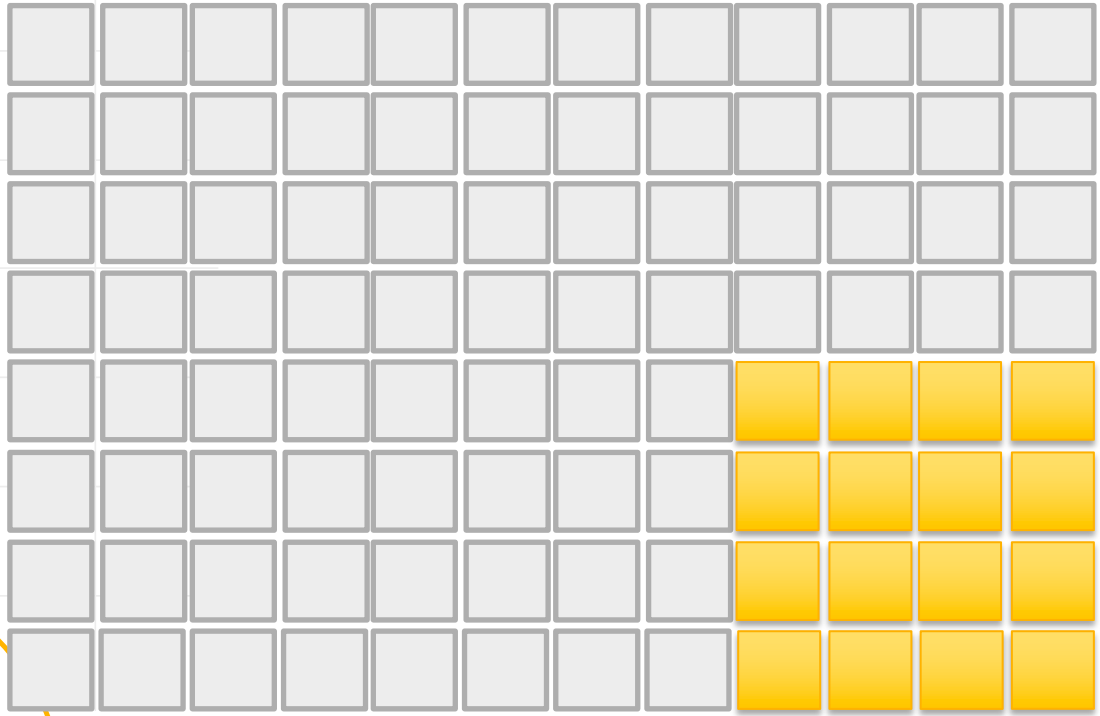
finding gcd(8,12)



If we remove a maximal square from the surface, S doesn't change

gcd(a,b) = gcd(a-b,b)

finding gcd(8,12)

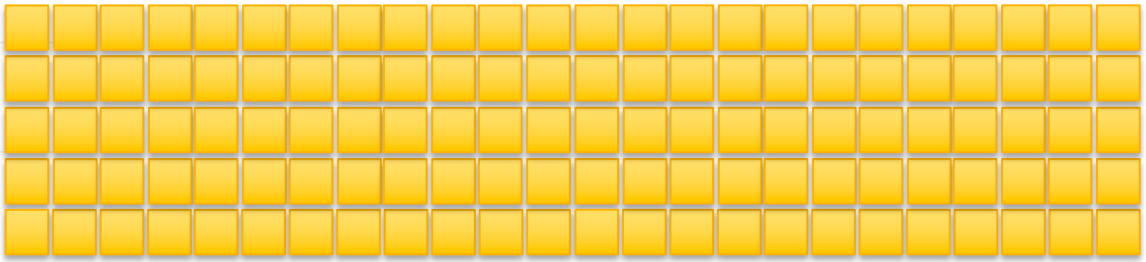


Repeat the process until we end up with a square.

a = b

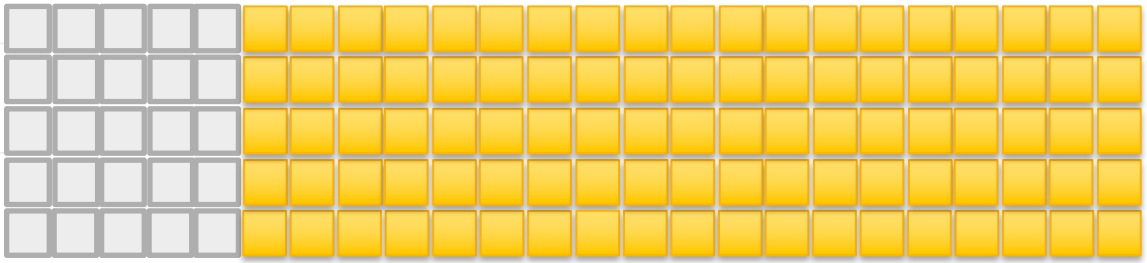
```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

5



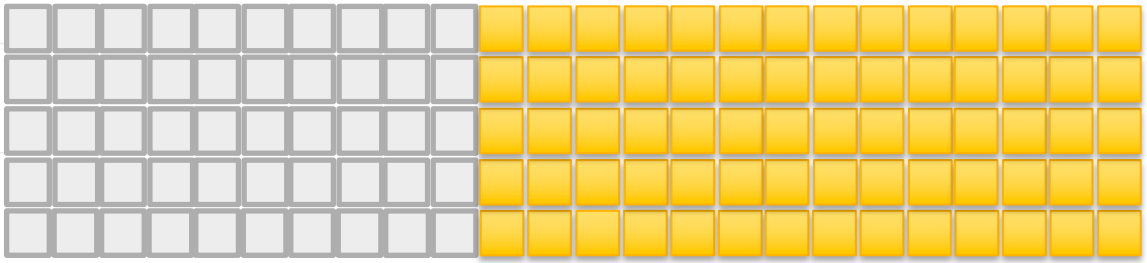
24

5



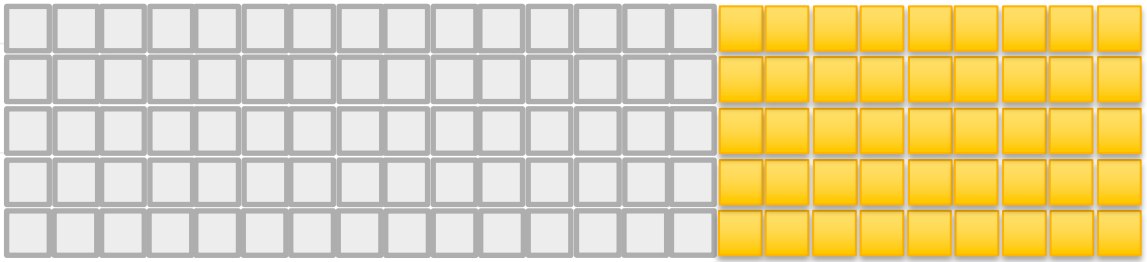
24

5



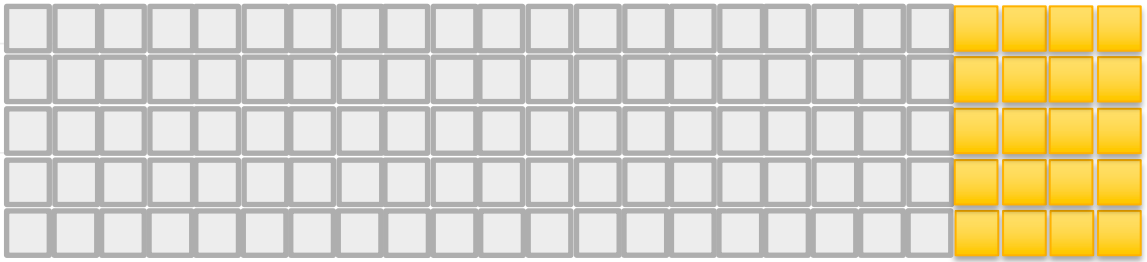
24

5



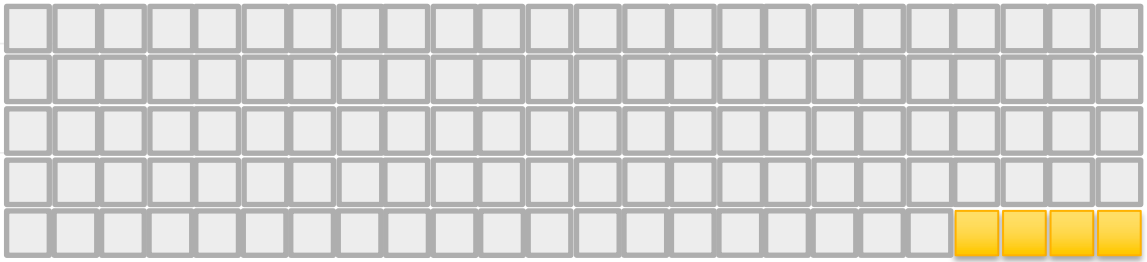
24

5



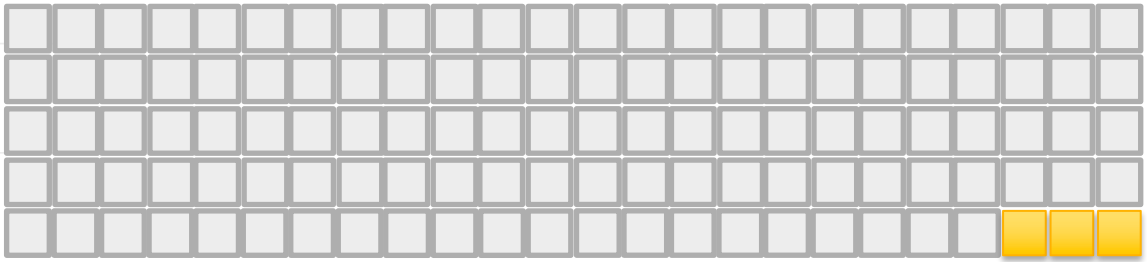
24

5



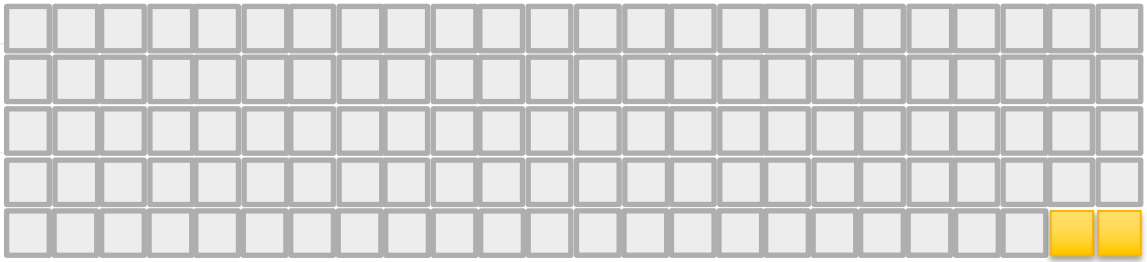
24

5



24

5



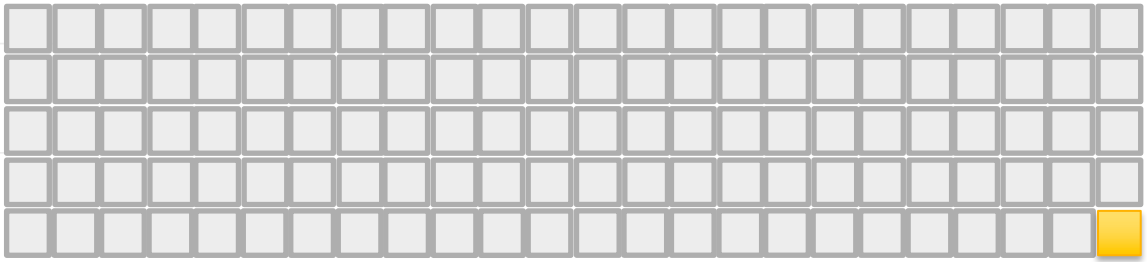
24

We can substantially improve the number of steps, if we replace:

gcd(a,b) = gcd(a-b,b)

by:

gcd(a,b) = gcd(a mod b,b)



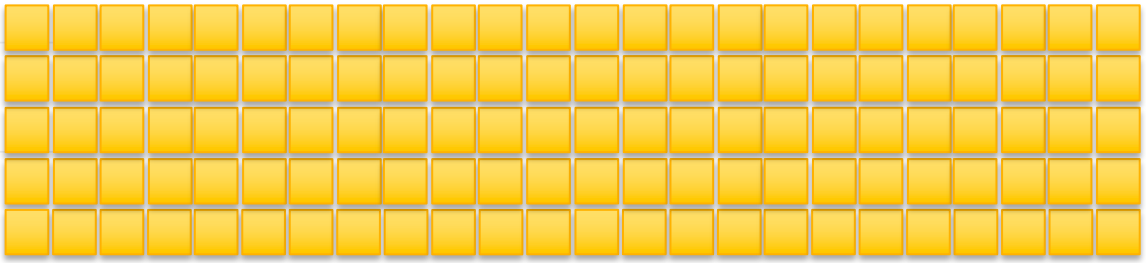
We can substantially improve the number of steps, if we replace:

$gcd(a,b) = gcd(a-b,b)$

by:

$gcd(a,b) = gcd(a \bmod b,b)$

5



24

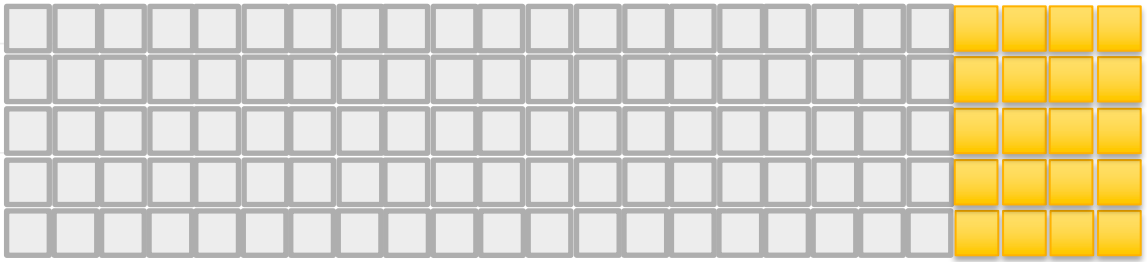
We can substantially improve the number of steps, if we replace:

gcd(a,b) = gcd(a-b,b)

by:

gcd(a,b) = gcd(a mod b,b)

5



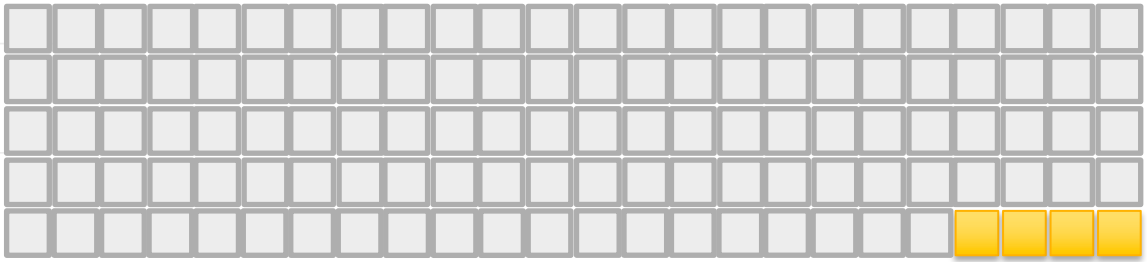
24

We can substantially improve the number of steps, if we replace:

gcd(a,b) = gcd(a-b,b)

by:

gcd(a,b) = gcd(a mod b,b)



5

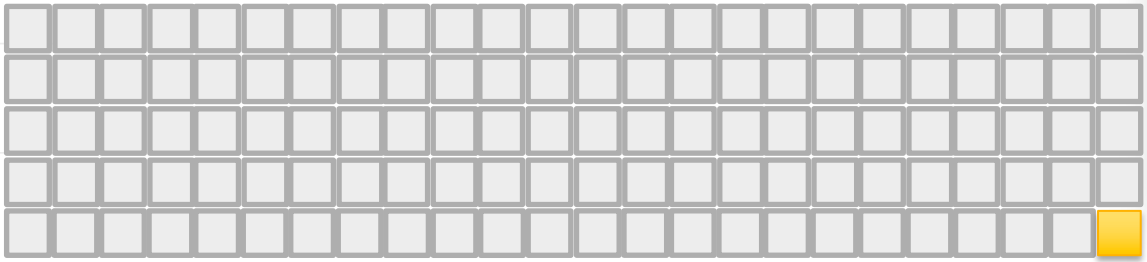
We can substantially improve the number of steps, if we replace:

$$gcd(a,b) = gcd(a-b,b)$$

by:

$$gcd(a,b) = gcd(a \bmod b,b)$$

```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```



5

Measuring the performance of subtraction gcd:

```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

Worst-case scenario:

b = 1 worst-case, we may have a subtractions

111111111… 11

Maximal value that we can store on n bits?

The size required to store a value n?

We determine how the number of instructions scale with respect to the size of the worst-case input.

Measuring the performance of subtraction gcd:

```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

Worst-case scenario:

b = 1

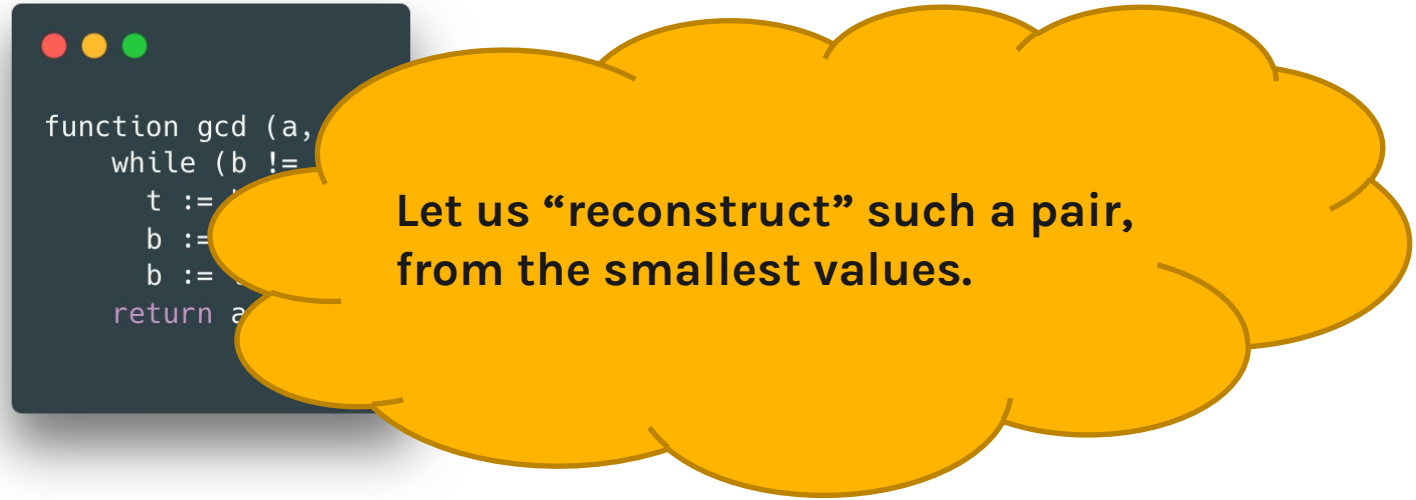
worst-case, we may have 2^n subtractions, where n is the number of bits required to store `max(a,b)`

We determine how the number of instructions scale with respect to the size of the worst-case input.

Measuring the performance of ***modulo*** gcd:

Worst-case scenario:

Pairs of numbers such that
`“a div b” = 1` (no speedup)



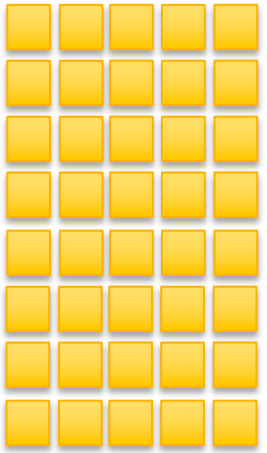
Measuring the performance of ***modulo*** gcd:

Worst-case scenario:

Pairs of numbers such that `"a div b" = 1` (no speedup)

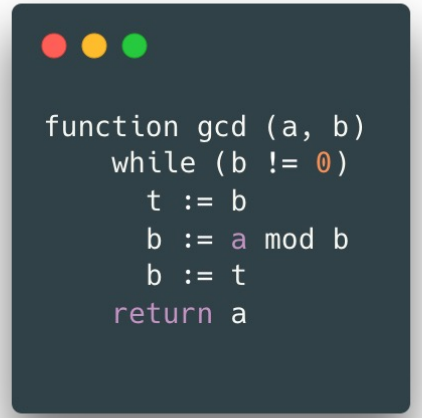
```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

These pairs are the Fibonacci numbers!



gcd(8,5)
gcd(5,3)
gcd(3,2)
gcd(2,1)
gcd(1,1)

Measuring the performance of ***modulo*** gcd:



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

Worst-case scenario:

gcd(f_{n+1}, f_n) will iterate n times over the while

$$f_{n+1} \simeq 1,6^{n+1}$$

$$f_{n+1} \leq 2^{n+2}$$

The number of bits required to store 2^{n+1} is n+1.

gcd will perform at most n iterations where max(a,b) is represented on n+1 bits.

subtraction gcd

modulo gcd



We need a language to facilitate expressing algorithm performance.

`Subtracti` at most 2^n subtractions input of size **n**.

`gcd` will perform at most **n** iterations for an input of size **n+1**.

`subtraction gcd` runs in exponential time

`modulo gcd` runs in linear time

An execution time for an algorithm **A** is a function $T : \mathbb{N} \to \mathbb{N}$ such that:

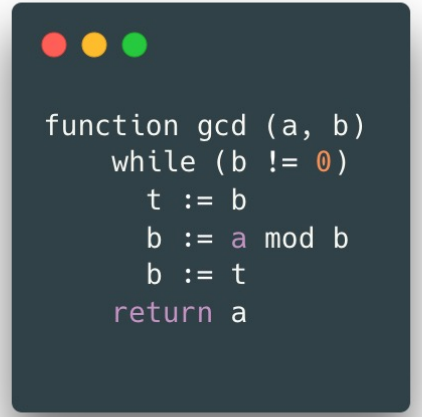
For any input of size n, $T(n) \geq$ the number of instructions performed by A on that very input

We expect T to be increasing (larger inputs, larger execution time)

We generally don't care about $T(n)$ for smaller n.

We are interested in how T increases as n increases, rather than the actual definition of T

We are interested in how `T` increases as `n` increases, rather than the actual definition of `T`

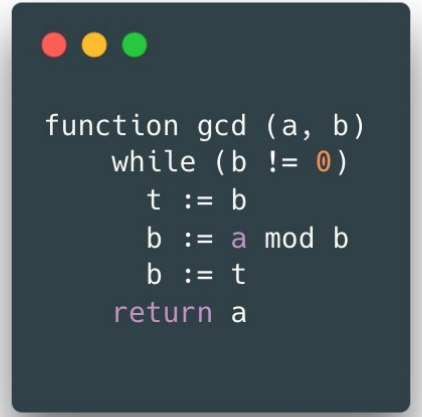


```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

$$T(n) = 4*(n-1) \text{ if } n > 1$$

$$T(n) = 1 \text{ otherwise}$$

We are interested in how `T` increases as `n` increases, rather than the actual definition of `T`



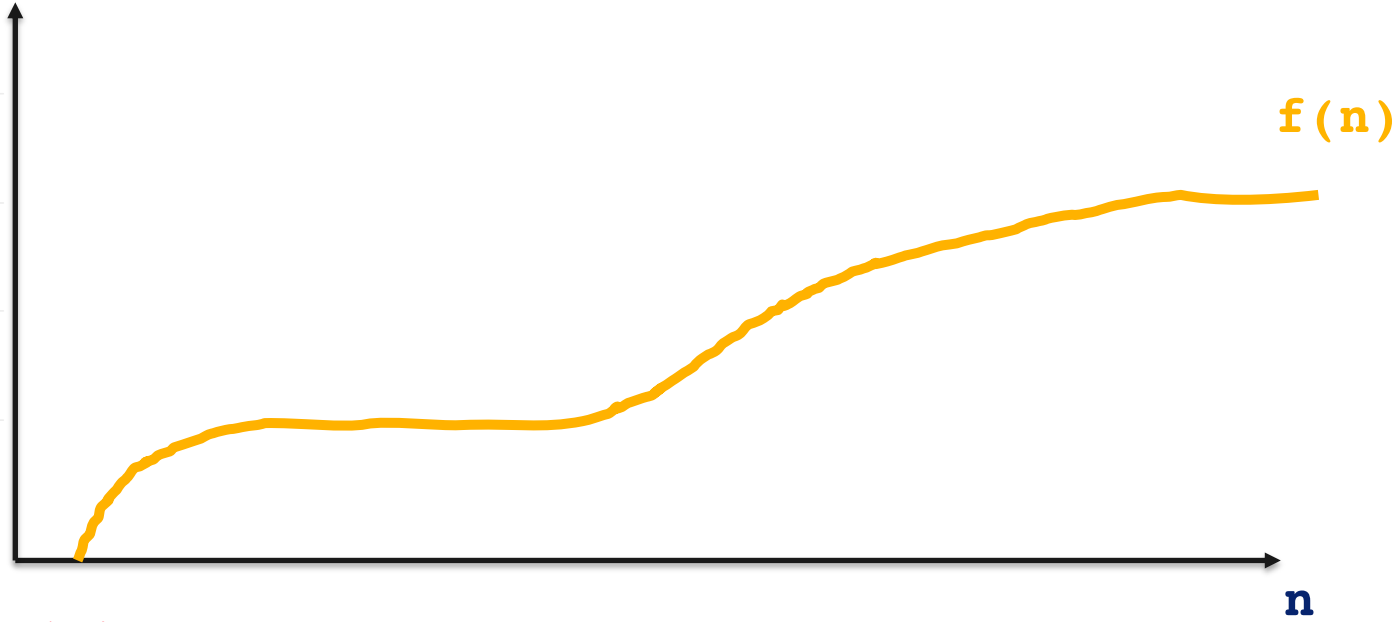
```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

$$T(n) = 4*(n-1) \text{ if } n > 1$$

`T` grows linearly with respect to the size of the input

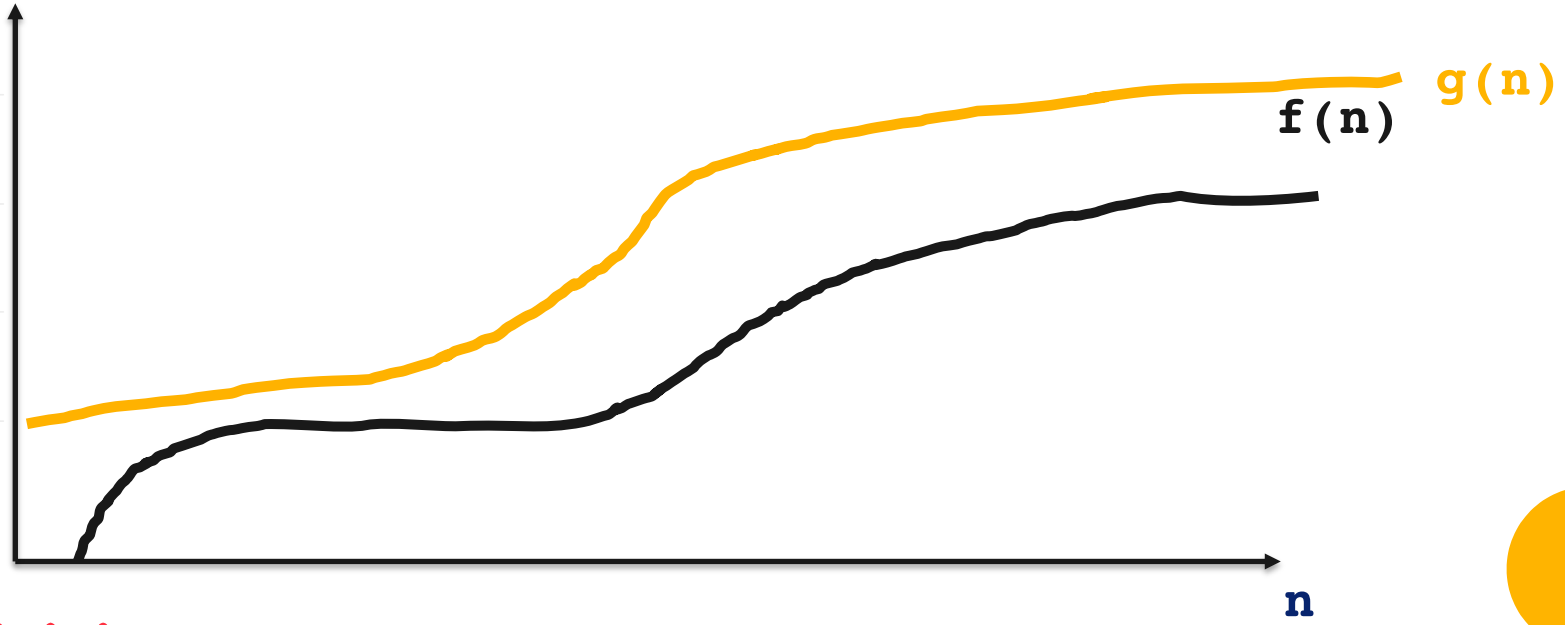
Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \exists c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



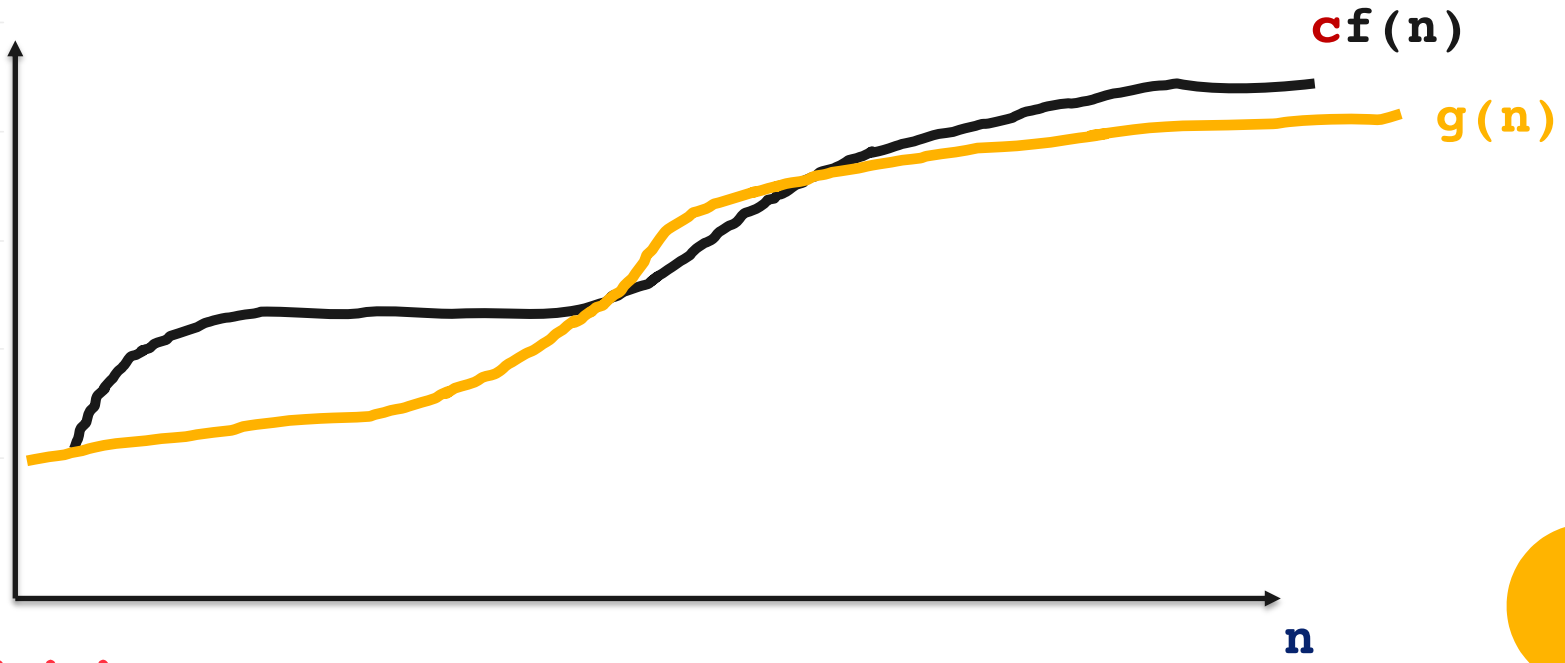
Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq g(n) \leq cf(n) \ \}$$



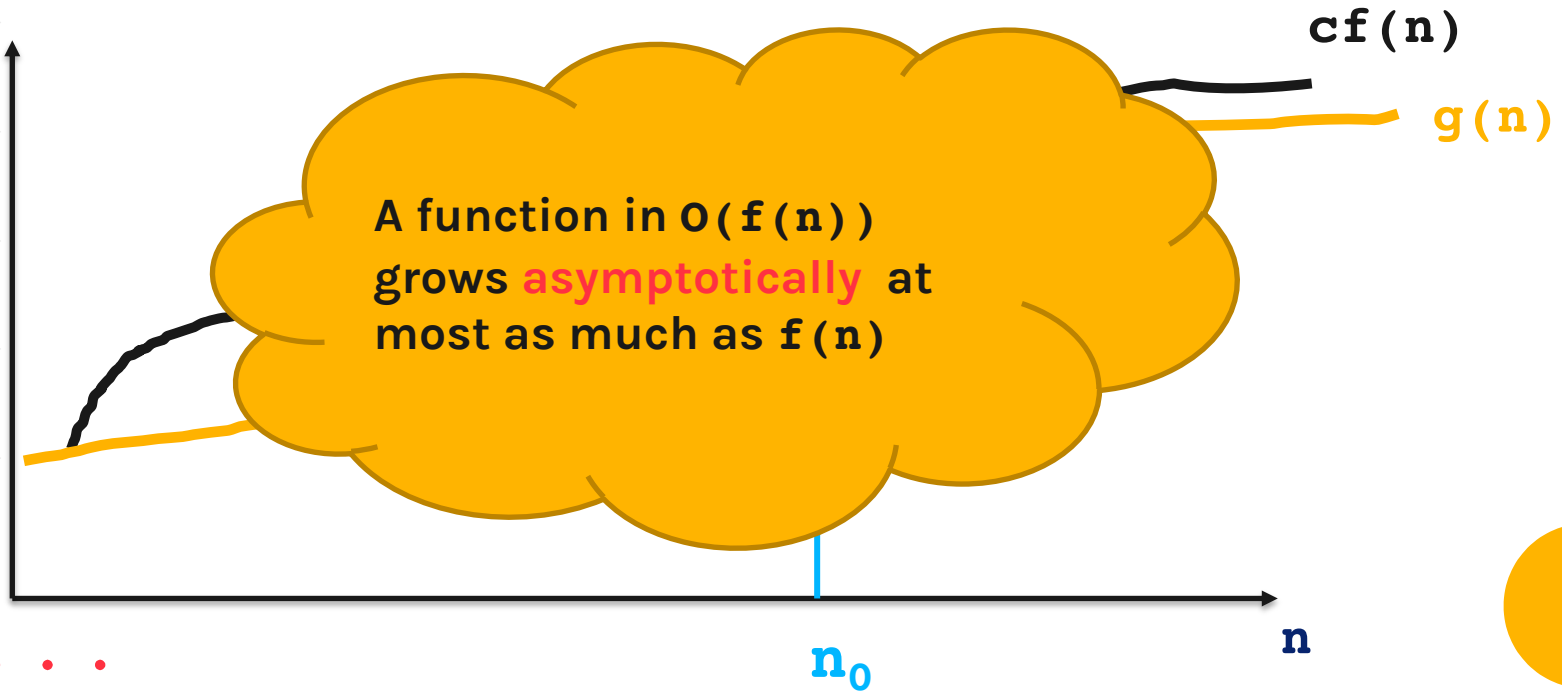
Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq g(n) \leq cf(n) \ \}$$



Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq g(n) \leq cf(n) \ \}$$



Big O notation

$O(\log(n))$

A class (or set) of functions

All members of $O(\log(n))$ grow asymptotically at most as $\log(n)$

We consider all members of $O(\log(n))$ to be equivalent w.r.t. how they grow

Saying that A has execution time $O(\log(n))$ means that A runs in logarithmic time, but we don't really care about its exact execution time.

Sometimes we care about the constant c. We may say that it is small, or give its exact value.

$T_{gcd_modulo}(n) \in O(n)$

$T_{gcd_modulo}(n) \in O(2n)$

Formally correct – doesn't tell us much more

$\log_2(n) \in O(n)$

true

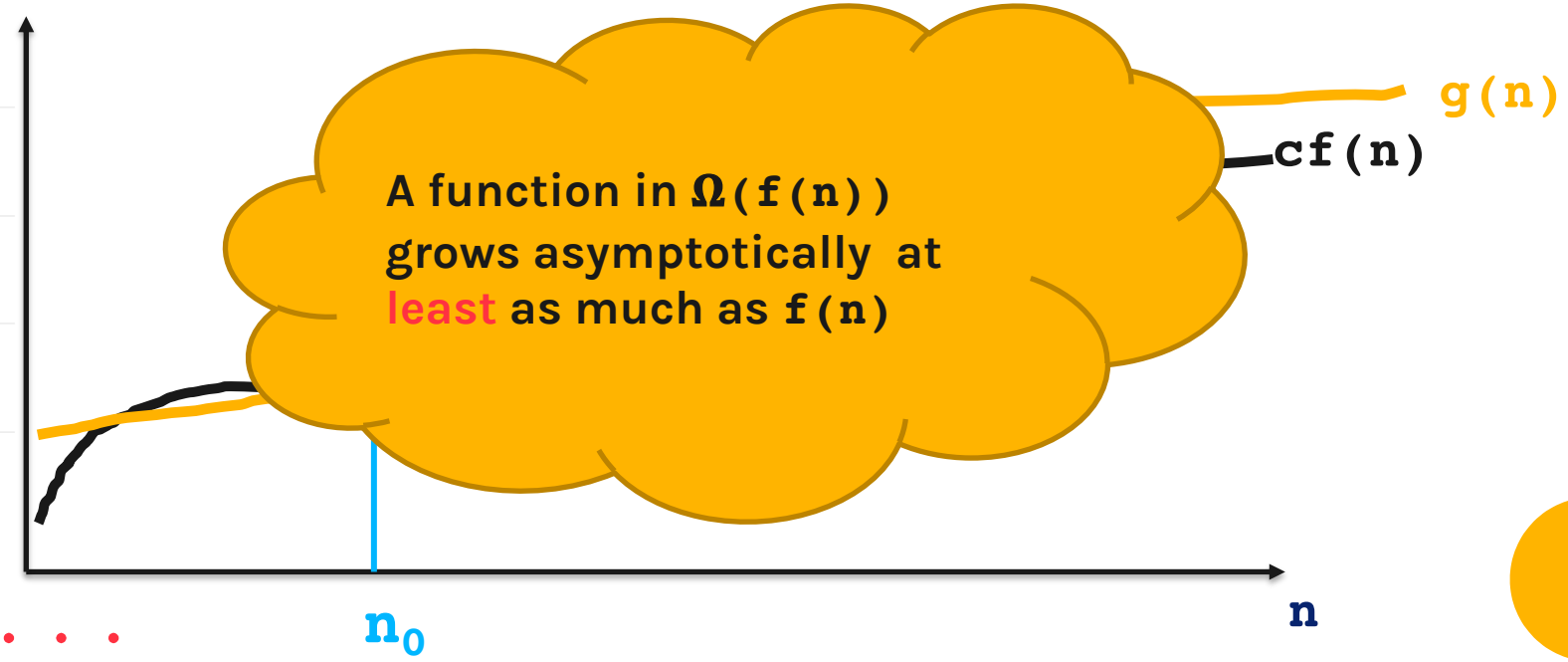
if $\lim_{n \to \infty} g(n)/f(n) = 0$ then

$g(n) \in O(f(n))$?

true

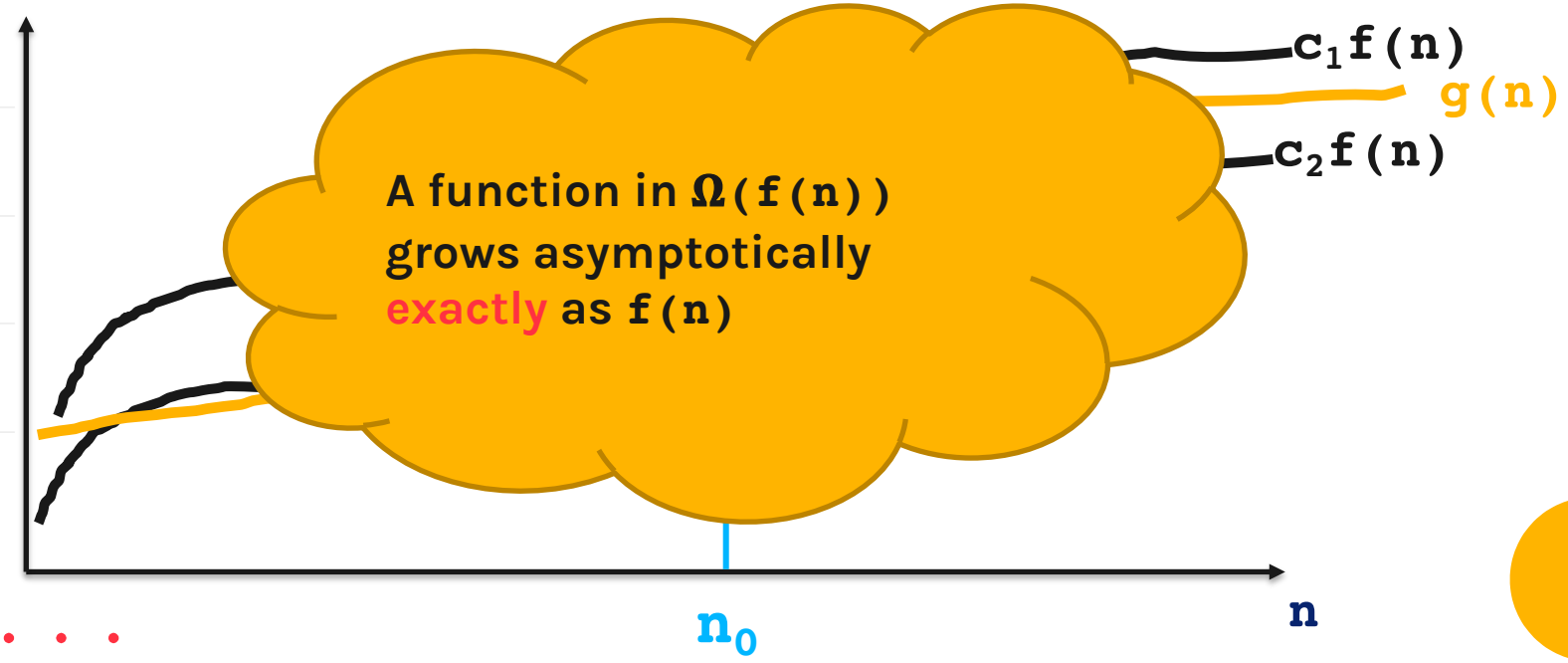
Omega (Ω) notation

$$\Omega(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq cf(n) \leq g(n) \ \}$$



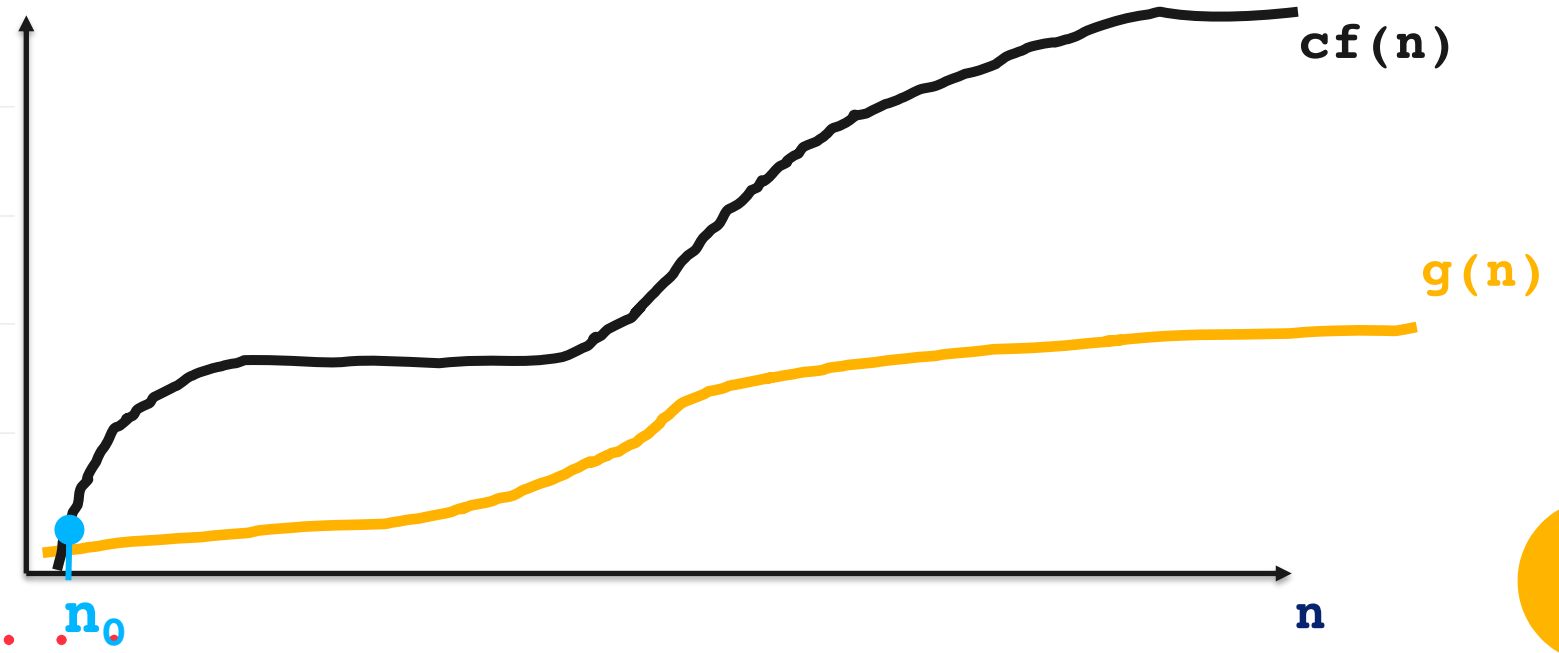
Theta (Θ) notation

$$\Theta(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+,\mathbb{R}^+) \;.\; \exists c_1,c_2 > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)\}$$



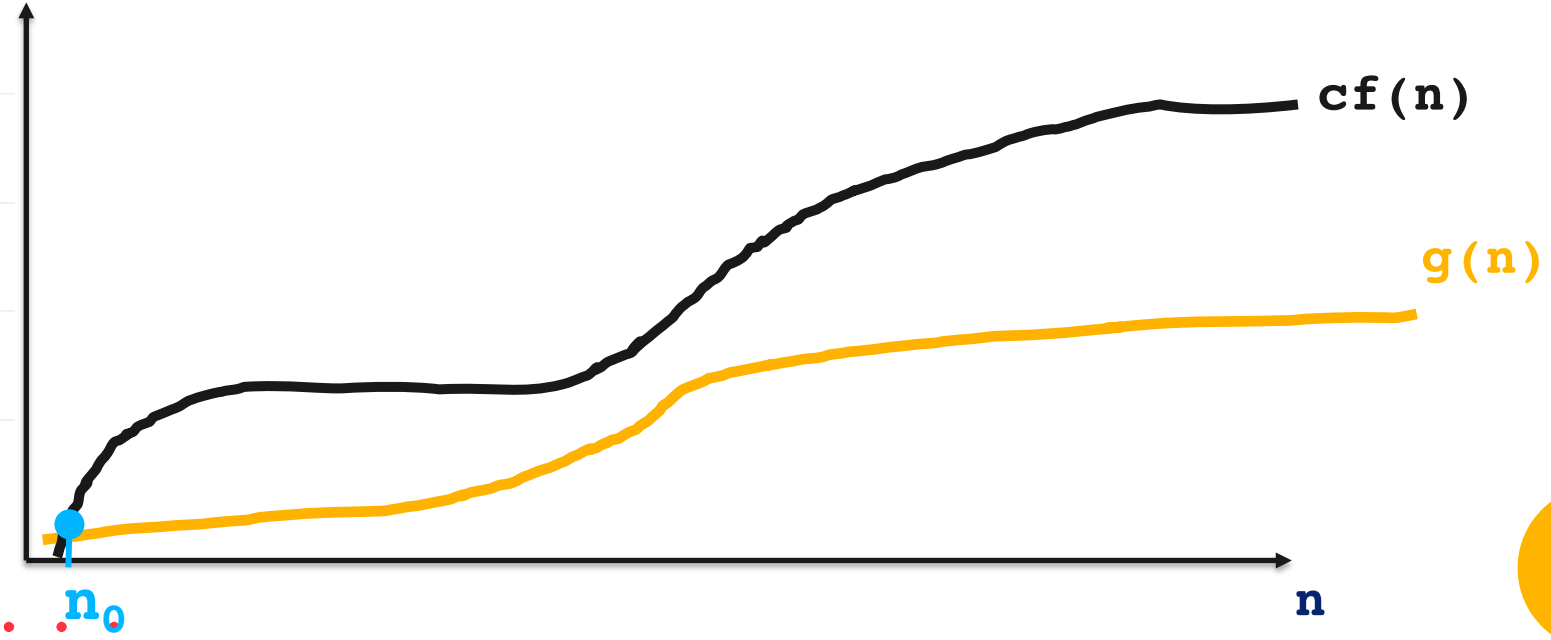
Small o notation

$$o(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \forall c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



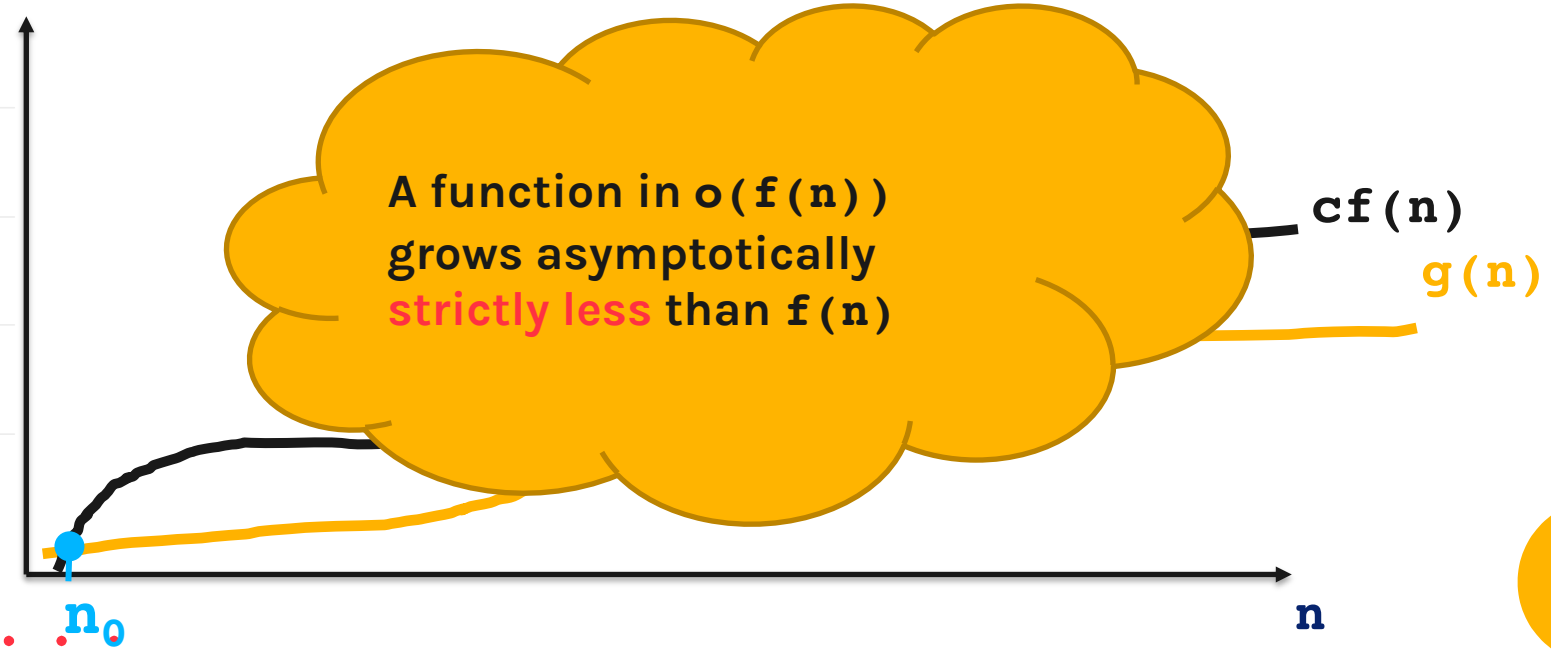
Small o notation

$$o(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \forall c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



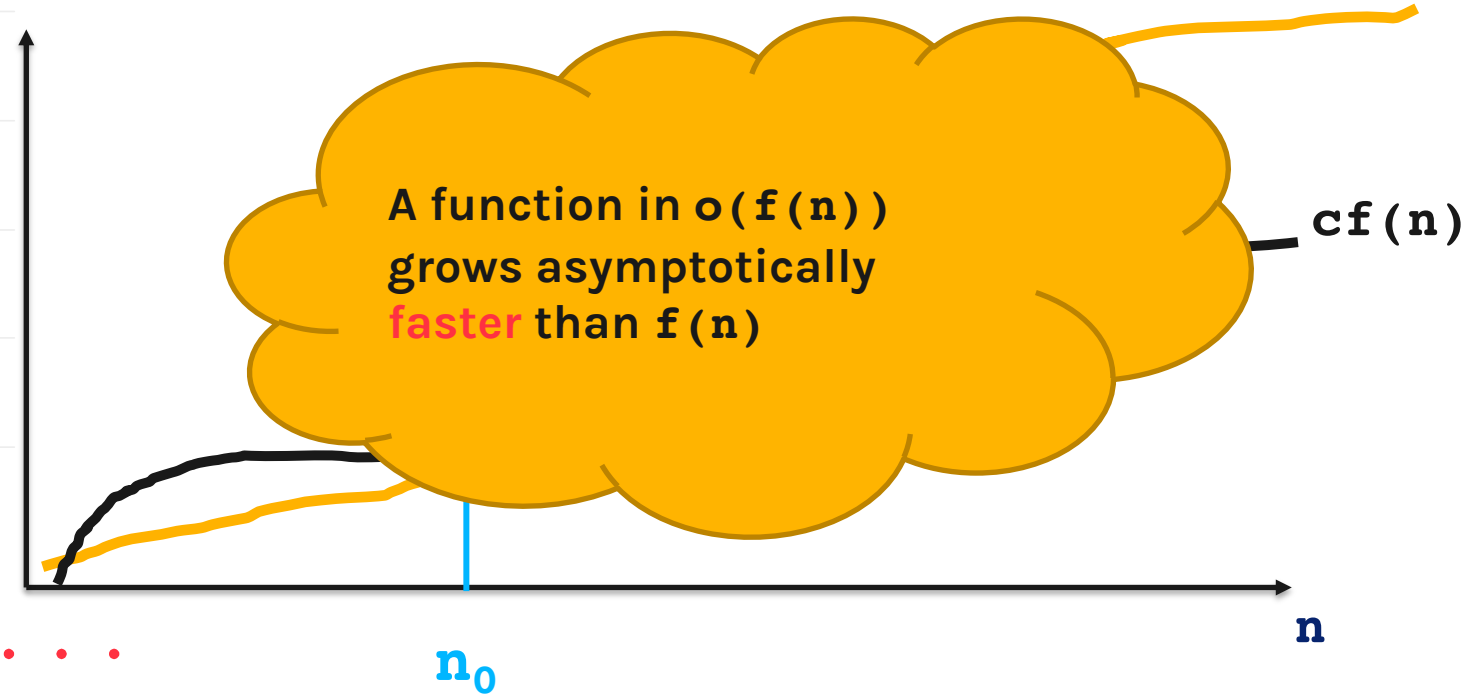
Small o notation

$$o(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \forall c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



Small ω notation

$$\omega(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \forall c > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, 0 \leq cf(n) \leq g(n)\}$$



Practice

$\log(n) \in \Omega(n)$?

$\text{sqrt}(n) \in \Omega(n)$?

if $f(n) \in O(n)$ then
$f(n) \in \Omega(n)$?

if $f(n) \in O(n)$ and $f(n) \in \Omega(n)$
then $f(n) \in \Theta(n)$?

Give an example of a function in $o(n)$

Give an example of a function in $o(n^2)$ and also in $\omega(n)$

Practice

Give an example of a function in $O(n)$ but not in $o(n)$ and neither in $\Theta(n)$.

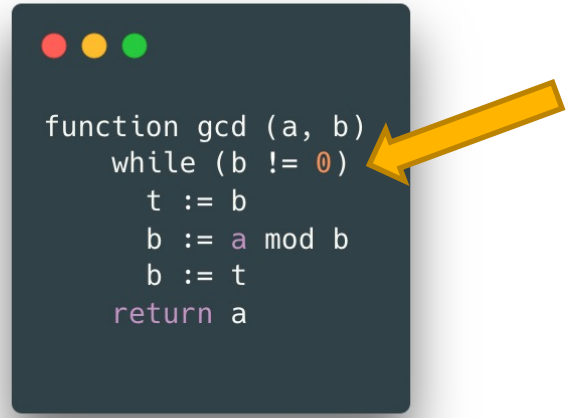
Practice

Give an example of a function in $\Omega(n)$ but not in $o(n)$ and neither in $\Theta(n)$.

Syntactic sugars

A formal language for
combining execution times

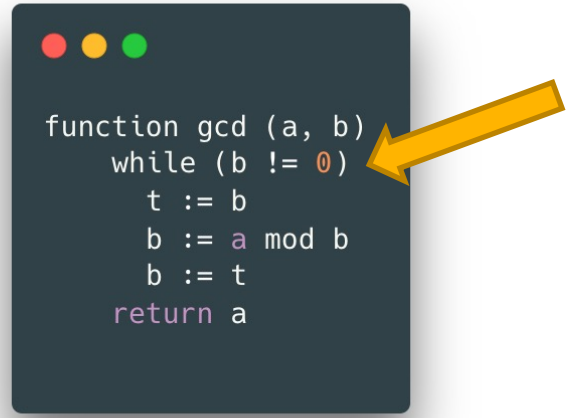
Syntactic sugars



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`n` or `n+1` iterations?

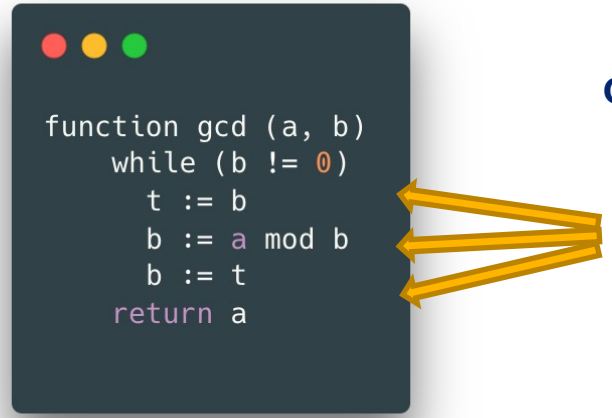
Syntactic sugars



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

Syntactic sugars

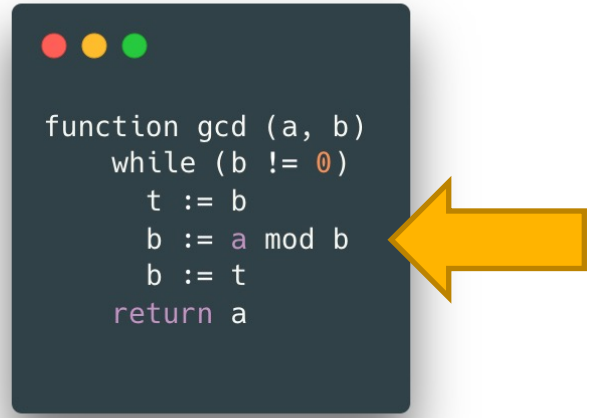


```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

three instructions?

Syntactic sugars

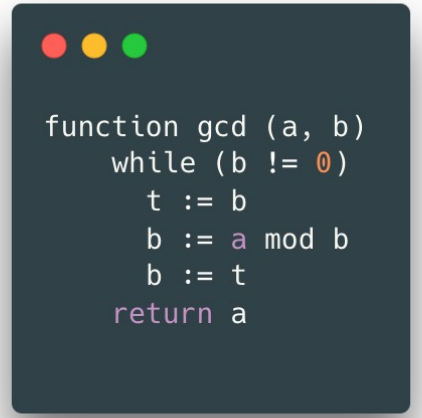


```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

`O(1)` (or constant) time

Syntactic sugars



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

`O(1)` (or constant) time

$$T(n) = O(n) * O(1) = O(n)$$

Syntactic sugars

$T(n) = O(n) * O(1) = O(n)$



Syntactic sugars

$$T(n) = O(n) * O(1) = O(n)$$

Syntactic sugars

$$T(n) = f(n) * O(1) = O(n)$$

$$f(n) \in O(n)$$

Syntactic sugars

$$T(n) = f(n) * g(n) = O(n)$$

$$f(n) \in O(n)$$

$$g(n) \in O(1)$$

Syntactic sugars

$T(n) = f(n) * g(n) = h(n)$

if $f(n) \in O(n)$ and $g(n) \in O(1)$

then $h(n) \in O(n)$

Practice with syntactic sugars

$O(n) * \Theta(\log n) = ?$

$\Theta(\log(n)) + \Theta(n) = ?$

$\Theta(n) / O(n) = ?$

$n*O(\log(n)) / \Theta(n) = ?$

$\Theta(n) / \omega(\sqrt{n}) = ?$

Lessons learned

Algorithm performance = how the consumed space asymptotically increases with respect to the size of the input, in the worst case.

Asymptotic notations & syntactic sugars = a rigorous language for talking about algorithm execution times.

Thank you!